

CSE Society — System Architecture & ID Specification

Official Design, Role Management, Notification Engine & Technical Implementation Documentation

1. CSE Society ID Format

Every member of the CSE Society will receive a unique Society ID. The ID will be automatically generated by the website when the student's membership is approved.

ID Format Structure

Example: **23F2601**

The Society ID has 4 parts:

- **23** → Admission Year: The last two digits of the year in which the student took admission. (Example: 2023 = 23)
- **F** → Current Academic Year:
 - F = 1st Year
 - S = 2nd Year
 - T = 3rd Year
 - R = 4th Year
- **26** → Membership Session Year (Example: Membership session 2026-27 = 26)
- **01** → Registration Sequence Number: This shows the order in which the student registered for the society membership.

Detailed Example

Suppose a student:

- Took admission in 2023
- Is currently in 1st Year
- Registers for the 2026-27 membership session
- Is the 1st student to register

Their Society ID will be: **23F2601**

Another student registering after them could receive: **23F2602**

Registration Sequence

The last two digits show the registration order:

- **01** → 1st registered member
- **02** → 2nd registered member
- **03** → 3rd registered member
- **10** → 10th registered member
- **25** → 25th registered member

This can be used to recognize early members of the society. For example, 01 can represent the first registered member for that particular category/session. However, the sequence number only shows registration order. It does not mean that the student is academically better or the "best student."

Automatic ID Generation

The Society ID should be generated automatically by the website, rather than being assigned manually.

The process will be:

Student Registration → Admin Verification → Approval → Automatic Society ID Generation → ID Saved in Database

This helps prevent:

- Duplicate IDs
- Wrong numbering
- Typing mistakes
- Missing sequence numbers
- Manual errors

Important Point

The Society ID should ideally be permanent. If a student's academic year changes from 1st year to 2nd year, their Society ID should not change automatically. The student's current year should be maintained separately in the database.

Final Pattern

YY + Y + SS + NN

Where:

- **YY** = Admission Year
- **Y** = Academic Year (F/S/T/R)
- **SS** = Membership Session Year
- **NN** = Registration Sequence Number (01-99)

Example: 23F2601

This makes the Society ID short, meaningful, easy to read, and easy to generate automatically.

2. Dashboard Architecture & Role Specifications

Final Dashboard Groups

Dashboard	Roles	Dashboard Characteristics
Admin Dashboard	President → renamed Admin	Unique
Executive Dashboard	Vice President + General Secretary	Exactly same
Secretary Dashboard	Technical Secretary + Cultural Secretary	Exactly same
Year Representative Dashboard	All Year Representatives	Exactly same
Member Dashboard	All normal members	Exactly same
Faculty In-charge Dashboard	2 CSE In-charges	Separate higher-authority dashboard

Important Distinction

Even when two roles share the same dashboard, we can still control what they are allowed to do through permissions.

For example:

- **Vice President + General Secretary** → Same interface → Same features → Same permissions
- **Technical Secretary + Cultural Secretary** → Same interface → Same features → Same permissions

Design Consistency

All dashboards will have a single common design system:

- Same colors
- Same fonts
- Same sidebar style
- Same navbar
- Same cards
- Same buttons
- Same icons
- Same spacing
- Same responsive design

Only the available features/options will change according to the role. So the website feels like one professional CSE Society platform, rather than several unrelated dashboards. And we'll keep the two CSE In-charges as a separate authority level above the student society roles.

Detailed Role Breakdown

Dashboard Group	Roles	Dashboard / Features
Faculty/Admin Dashboard	2 CSE In-charges + Admin (President)	Exactly same
Executive Dashboard	Vice President + General Secretary	Exactly same
Secretary Dashboard	Technical Secretary + Cultural Secretary	Exactly same
Year Representative Dashboard	All Year Representatives	Exactly same
Treasurer Dashboard	Treasurer	Separate, simple
Member Dashboard	All Normal Members	Exactly same

1. Faculty/Admin Dashboard

CSE In-charges + Admin: All three will have the same dashboard, same features, same options, and same interface. The dashboard can contain:

Dashboard, Members, Executive Committee, Events, Registrations, Attendance, Announcements, Certificates, Reports, Society Activities, Profile

2. Executive Dashboard

Vice President + General Secretary: Exactly the same:

Dashboard, Members, Events, Registrations, Attendance, Announcements, Activities, Reports, Profile

3. Secretary Dashboard

Technical Secretary + Cultural Secretary: Exactly the same:

Dashboard, Events, Registrations, Attendance, Activities, Announcements, Resources, Reports, Profile

The dashboard isn't technically different based on whether the person is Technical or Cultural Secretary.

4. Year Representative Dashboard

All 1st, 2nd, 3rd and 4th Year Representatives: Exactly the same dashboard:

Dashboard, Students, My Year, Events, Attendance, Announcements, Feedback, Reports, Profile

The dashboard remains identical; only the representative's associated year changes.

5. Treasurer Dashboard

A separate and intentionally very simple dashboard:

Dashboard, Income, Expenses, Transactions, Budget, Reports, Profile

6. Member Dashboard

All normal members get exactly the same dashboard:

Dashboard, My Profile, My Society ID, Events, My Registrations, Attendance, Certificates, Announcements, Resources, Feedback

Important Rule: Same role group = 100% same dashboard.

For example:

- **CSE In-charge A = CSE In-charge B = Admin:** Same dashboard + same features + same options
- **Vice President = General Secretary:** Same dashboard + same features + same options
- **Technical Secretary = Cultural Secretary:** Same dashboard + same features + same options
- **1st Year Rep = 2nd Year Rep = 3rd Year Rep = 4th Year Rep:** Same dashboard + same features + same options

3. Notification System Specification

The CSE Society website will have a central notification system available across all dashboards.

1. Notification Panel

Every user will have a  Notification Panel. It will show the latest notifications received by that user. Each notification can display:

- Title
- Short message
- Sender
- Date & time
- Read/Unread status

Users can open a notification to view its complete message.

2. Notification History

There will also be a separate **Notifications** → **History** section containing the complete history of notifications received by the user. Users can:

- View old notifications
- Search notifications
- See date and time
- See sender
- Identify read/unread notifications

3. Sending Notifications

All society roles except Normal Members will have a **Send Notification** option. The sender can select the recipient.

Recipient options:

- All
- Vice President
- General Secretary
- Treasurer
- Technical Secretary

- Cultural Secretary
- Year Representative – 1st Year
- Year Representative – 2nd Year
- Year Representative – 3rd Year
- Year Representative – 4th Year
- Specific Member

For a specific member, the sender can enter their CSE Society ID (Example: **CSE ID: 23F2601**). The system will identify that member and send the notification specifically to them.

4. Notification Flow

Sender → Send Notification → Select Recipient (All / Role / Year Representative / Specific CSE ID) → Write Notification (Title + Message) → Send → Recipient's Notification Panel → Notification History

5. Normal Member Rights

Normal members will not have the Send Notification option. They will only have  Notification Panel and  Notification History. They can view all notifications received by them and access their previous notification history.

6. Notification Record Structure

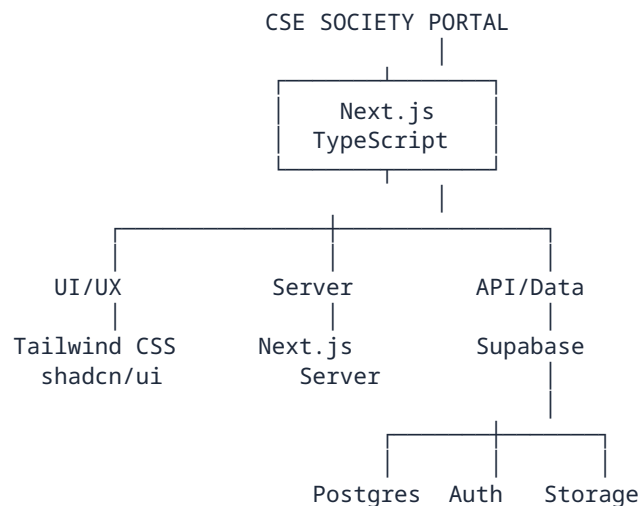
Each notification should maintain:

- Notification ID
- Sender
- Sender's role
- Recipient
- Recipient type
- Title
- Message
- Date & time
- Read/Unread status

4. Recommended Technical Architecture & Stack

Overall Architecture

Next.js + TypeScript + Tailwind CSS + shadcn/ui + Supabase + Vercel



Stack Details

1. Frontend — Next.js + TypeScript

Recommended: Next.js (App Router) + TypeScript

Next.js is a React framework for building full-stack applications. Its App Router supports modern React features and server-side rendering. For your project, this means you can keep the website, dashboards, server-side logic and API functionality in one application instead of creating a separate frontend and backend unnecessarily.

Why Next.js?

- **Fast page navigation:** Next.js automatically performs route-based code splitting and prefetching, which can make navigation feel very fast.
- **Server-side rendering:** You can render appropriate parts of dashboards on the server instead of sending everything to the browser.
- **Good for dashboards:** Your project has Login, Multiple dashboards, Member profiles, Events, Notifications, Tables, Forms, Reports. These are all very suitable for Next.js.
- **Scalable:** You only have around 350 students, but if the society grows to thousands of users, the architecture will still be suitable.

2. TypeScript

I strongly recommend TypeScript instead of plain JavaScript. It helps prevent mistakes such as `studentId`, `studentID`, `societyId`, `societyID` being accidentally treated as different things in your code. For a system containing Students, Roles, Events, Notifications, Transactions, Certificates, Society IDs, TypeScript will make the project much easier to maintain.

3. UI — Tailwind CSS

Use Tailwind CSS for styling. It is particularly suitable for your requirement because you want all dashboards to have the same design and color system. We can create a common design system (Primary Color, Secondary Color, Background, Cards, Buttons, Sidebar, Navbar, Text, Borders, Status Colors) then every dashboard automatically follows the same visual language.

4. UI Components — shadcn/ui

I recommend shadcn/ui for components such as Buttons, Dialogs, Dropdowns, Tables, Forms, Tabs, Cards, Notifications, Sidebars, Modals, Select menus. This will help us keep all dashboards consistent. For example, Admin Dashboard and Member Dashboard can use exactly the same underlying UI components while showing different content.

5. Database — PostgreSQL through Supabase

This is the part I especially recommend for your project. Use Supabase PostgreSQL. Supabase provides a full PostgreSQL database rather than an abstraction over PostgreSQL. It also integrates with authentication, storage and realtime functionality.

Why PostgreSQL? Your data is naturally relational.

```
Student → Society ID → Membership → Event Registration → Attendance → Certificate  
User → Role → Dashboard → Notifications
```

A relational database is ideal for this.

6. Database Size

You said: Maximum 350 students. That's extremely manageable for PostgreSQL. You could have 350 students, Executive members, Faculty, Hundreds/thousands of notifications, Hundreds of events/registrations, Certificates, Attendance records, Financial transactions, Audit/history records and PostgreSQL would still be a very comfortable choice. In fact, I would not choose a NoSQL database simply because the project is small. PostgreSQL gives you strong relationships and structured data, which is exactly what your system needs.

7. Authentication — Supabase Auth

Use Supabase Auth instead of building your own authentication system from scratch. Supabase Auth supports common authentication methods and integrates with PostgreSQL. It also supports JWT-based authentication and can work with Row Level Security. For your website, you could initially use Email + Password and potentially add other methods later if required. The user account would be connected to the student's society profile:

```
Authentication Account → User Profile → Society Member → Society ID
```

8. Notifications — Supabase Realtime

This is particularly useful because you've already decided that the website needs a notification panel and notification history. Supabase Realtime can listen to database changes and deliver updates to connected clients. Supabase specifically supports real-time notifications and live dashboards.

So when an authorized user sends a notification:

```
Sender → Notification created in PostgreSQL → Supabase Realtime → Recipient's browser  
→ 🔔 New Notification
```


The user doesn't need to manually refresh the page to see a new notification. For a system like yours, this is a very good fit.

9. File Storage — Supabase Storage

You'll eventually need to store Profile photos, Society documents, Certificates, Event images, PDFs, Other society files. Supabase includes Storage alongside its database and authentication services. So you don't need to create a separate file-storage backend initially.

10. Backend — Don't Build a Separate Express Server Initially

For this particular project, I wouldn't start with React → Express.js → PostgreSQL unless you have a specific reason to need a separate backend.

Instead: Next.js → Server-side logic / API → Supabase → PostgreSQL

Next.js is already designed for full-stack applications. This means fewer moving parts: No unnecessary separate backend deployment.

11. Deployment — Vercel

For deployment, I'd recommend Vercel, especially because you're using Next.js. Vercel provides first-class support for Next.js, including CDN delivery for static assets and serverless handling for server-rendered/API functionality. Your architecture becomes:

```
GitHub → Vercel → Next.js Website → Supabase → PostgreSQL
```

12. GitHub

Use GitHub for Source code, Version control, Collaboration, Backup of project code, Deployment integration. You can work with main, development, feature/* branches if multiple people eventually work on it.

13. Database Structure Overview

I would start with tables roughly like these:

users, members, society_ids, roles, executive_members, events, event_registrations, attendance, notifications, notification_recipients, certificates, announcements, income, expenses, transactions, documents, gallery, audit_logs

We don't need to create every table immediately. We can design the database properly after we finalize all website features.

14. Notification Database Design

For your notification system, I would not simply store one message in a single column like notification → "message". Instead, we should have something like:

```
notifications:
  id, title, message, sender_id, created_at

notification_recipients:
  id, notification_id, recipient_id, read, read_at
```

This allows one notification to go to All Members, or All Year Representatives, or One specific Society ID while still maintaining individual read/unread status.

15. Performance Best Practices

For your requirement of fast response, the stack is more than sufficient. But the technology alone doesn't guarantee speed. We should also follow good practices:

- **Database:** Proper indexes, Pagination, Efficient queries, Avoid fetching unnecessary columns
- **Next.js:** Server rendering where useful, Route-based code splitting, Caching where appropriate, Optimized images
- **Frontend:** Don't load the entire database at once, Use pagination for member lists, Lazy-load large sections, Avoid unnecessary client-side state

16. Summary Requirement-to-Technology Mapping

Requirement	Technology
Website	Next.js
Programming language	TypeScript
UI styling	Tailwind CSS
UI components	shadcn/ui
Database	PostgreSQL
Database platform	Supabase
Authentication	Supabase Auth
Notifications	Supabase Realtime
File storage	Supabase Storage
Backend/server logic	Next.js
Code management	GitHub
Deployment	Vercel

The Biggest Advantage: You aren't creating five different systems. You're creating one unified application.

For a maximum of 350 students, this is more than enough. It gives you a professional architecture without unnecessarily introducing Redis, Docker, Kubernetes, a separate Express backend, or multiple databases. And importantly, it leaves room to expand the system later if the CSE Society grows.

Final Recommendation: Next.js + TypeScript + Tailwind CSS + shadcn/ui + Supabase (PostgreSQL + Auth + Storage + Realtime) + Vercel + GitHub.